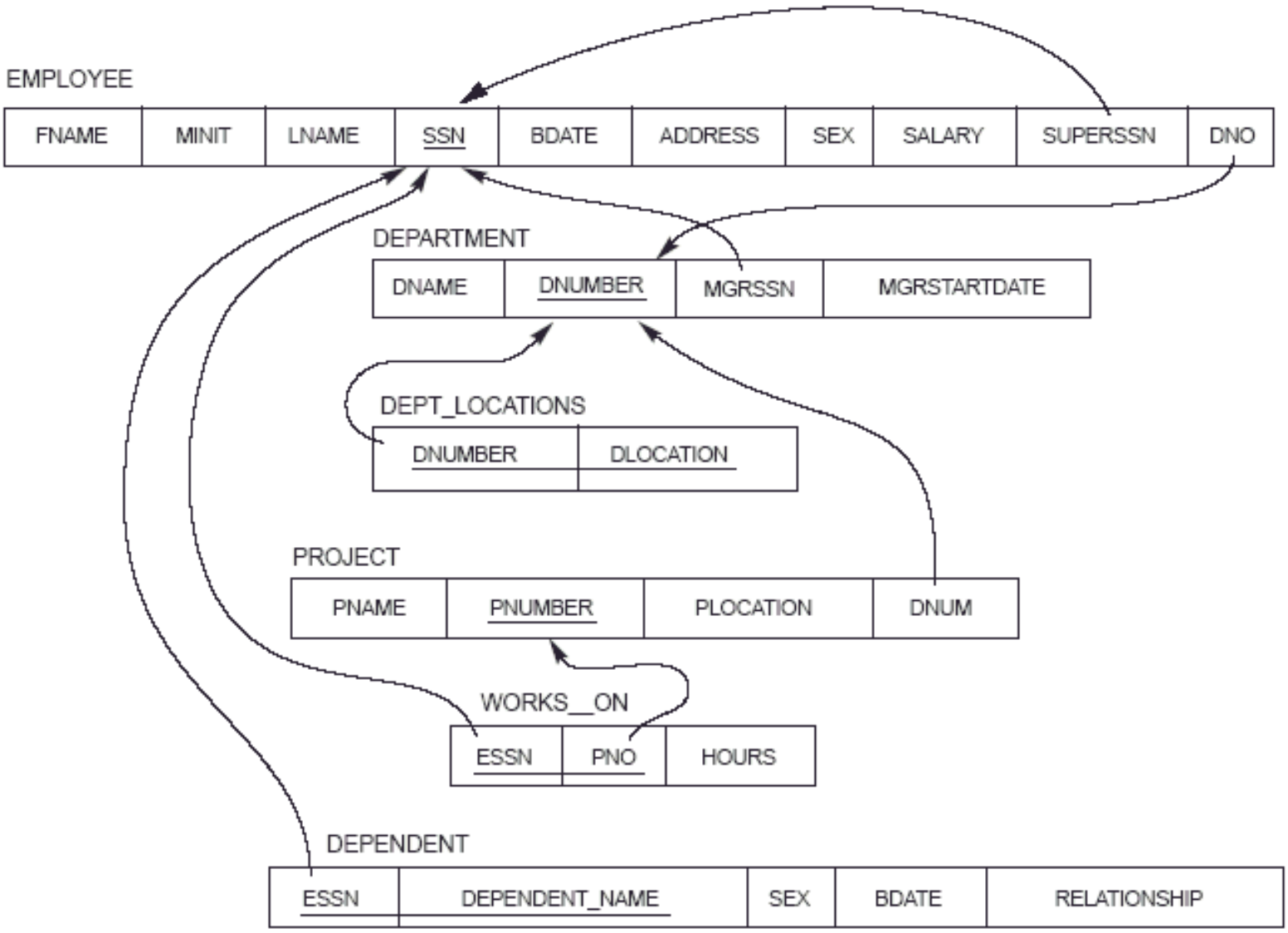
SQL – The Relational Database Standard

Figure 7.7 Referential integrity constraints displayed on the COMPANY relational database schema diagram.



SQL

- Why SQL?
  - SQL is a very-high-level language, in which the programmer is able to avoid specifying a lot of data-manipulation details that would be necessary in languages like C++.

SQL

- SQL is a comprehensive database language; it has statements for data definition, query, and update. Hence, it is both DML and DDL.

SQL

- Defining a relation schema in SQL
  - Data definition: describing the structure of information in the database.
  - Data manipulation: queries and modifications

SQL

The different DDL commands are as follows.

- CREATE TABLE
- ALTER TABLE …….ADD
- ALTER TABLE…..MODIFY
- DROP TABLE

SQL

- Data types
  - Character strings of fixed varying length.
    - CHAR(n): a fixed-length string of n characters.
    - VARCHAR(n): a string of up to n characters
  - Bit strings of fixed or varying length.
    - BIT(n): bit strings of length n
    - BIT VARYING(n): bit strings of length up to n.
  - BOOLEAN
    - Denote an attribute whose value is logical.
    - TRUE, FALSE, UNKNOWN

SQL

- INT
  - INT and INTEGER are synonyms
  - SHORTINT: the number of bits may be less
- Floating-point numbers
  - FLOAT and REAL are synonyms
  - DOUBLE PRECISION: higher precision
  - DECIMAL(n,d): consists of n decimal digits, with the decimal point assumed to be d positions from the right. E.g. 0123.45 is in DECIMAL(6,2)
  - NUMERIC is a synonym for DECIMAL

SQL

- Dates and times
  - DATE and TIME: essentially, they are character strings of a special form.

Figure 7.7 Referential integrity constraints displayed on the COMPANY relational database schema diagram.

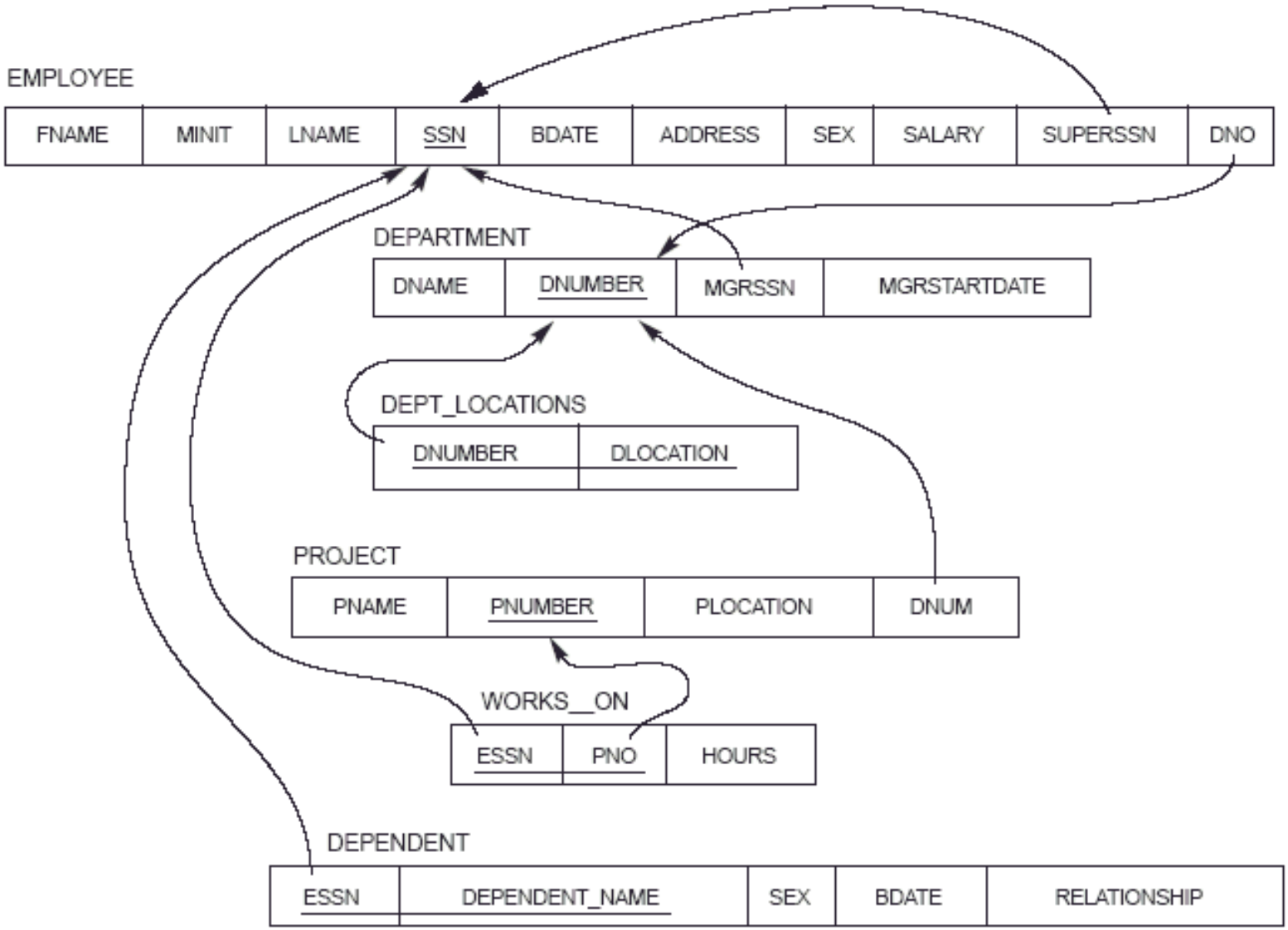


Figure 5.6

One possible database state for the COMPANY relational database schema.

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

SQL

- CREATE TABLE.....

  This is used to create a new relation and the corresponding syntax

  CREATE TABLE relation_name
  (field 1 data type(size), field 2 data type(size), ..., field n data type(size));

SQL

- Example :

```
CREATE TABLE EMPLOYEE
(  FNAME        VARCHAR(15)       NOT NULL,
   MINT         CHAR,
   LNAME        VARCHAR(15)       NOT NULL,
   SSN          CHAR(9)      NOT NULL,
   BDATE        DATE,
   ADDRESS      VARCHAR(30),
   SEX          CHAR,
   SALARY              DECIMAL(10,2),
   DNO          INT     NOT NULL );
```

SQL

- A relation may have only one primary key. Two ways to declare:
  - Declare one attribute to be a primary key when that attribute is listed in the relation schema.
  - Add to the list of items declared in the schema an additional declaration that says a particular attribute or set of attributes forms the primary key.

SQL

- For 1), the keywords PRIMARY KEY should be added after the attribute and its type
- For 2), the keywords PRIMARY KEY and a parenthesized list of the attribute or attributes that form this key.
- If the key consists of multiple attributes, method 2) need to be used.

SQL

- Distinctions between UNIQUE and PRIMARY KEY:
  - We may have any number of UNIQUE declarations for a table, but only one primary key.
  - PRIMARY KEY forbids NULL's while UNIQUE permits them.

SQL

- Declare an attribute or attributes of one relation to be a foreign key, referencing some attribute(s) of a second relation.
  - The referenced attribute(s) of the second relation must be declared UNIQUE or the PRIMARY KEY for their relation
  - Values of the foreign key appearing in the first relation must also appear in the referenced attributes of some tuples.

SQL

- Two ways:
  - If the foreign key is a single attribute, we may follow its name and type by a declaration that it “references” some attribute of some table.

    REFERENCES <table> (<attribute>)
  - Alternatively, append to the list of attributes in a CREATE TABLE statement one or more declarations stating that a set of attributes is a foreign key.

    FOREIGN KEY (<attributes>) REFERENCES <table> (<attribute>)

SQL

- Example :

```
CREATE TABLE EMPLOYEE
  ( FNAME          VARCHAR(15)       NOT NULL,
    SSN            CHAR(9)      NOT NULL,
    .................
    DNO            INT     NOT NULL,
    PRIMARY KEY(SSN),
    FOREIGN KEY(SUPERSSN) REFERENCES EMPLOYEE(SSN),
    FOREIGN KEY(DNO) REFERENCES DEPARTMENT(DNUMBER)
   );
```

SQL

- A *constraint* is a relationship among data elements that the DBMS is required to enforce.
  - Example: key constraints.

SQL

- The options of policies are stated with the declaration of the foreign key.
- ON DELETE or ON UPDATE followed by our choice of SET NULL or CASCADE.

SQL

- Example :

```
CREATE TABLE EMPLOYEE
  ( ..................
    DNO                INT     NOT NULL DEFAULT 1,
    CONSTRAINT EMPPK
      PRIMARY KEY(SSN),
    CONSTRAINT EMPSUPERFK
      FOREIGN KEY(SUPERSSN) REFERENCES      EMPLOYEE(SSN)
    ON DELETE SET NULL ON UPDATE         CASCADE,
    CONSTRAINT EMPDEPTFK
      FOREIGN KEY(DNO) REFERENCES
      DEPARTMENT(DNUMBER) ON DELETE SET DEFAULT         ON
    UPDATE CASCADE
  );
```

SQL

- ALTER TABLE.....ADD....

  This is used to add some extra fields into an existing relation. The generalized format is given

  ALTER TABLE ADD relation_name
  ADD (new field_1 data type(size),
  new field_2 data type(size),
  .....);

SQL

- Example :

```
ALTER TABLE COMPANY.EMPLOYEE
        ADD JOB VARCHAR(12);

ALTER TABLE EMPLOYEE
        ADD JOB VARCHAR(12);
```

SQL

ALTER TABLE.....MODIFY....

This is used to change the width as well as data type of fields of existing relations. The generalized format is given

ALTER TABLE ADD relation_name

ALTER (field_1 new data type(size),

field_2 new data type(size),

.....);

SQL

- Example

```
ALTER TABLE DEPARTMENT
  ALTER MGRSSN DROP DEFAULT;

ALTER TABLE DEPARTMENT
  ALTER MGRSSN SET DEFAULT '333445555';
```

SQL

- DROP TABLE :

  This is used to delete a relation. The generalized syntax is

  DROP TABLE relation_name

Example

DROP TABLE DEPENDENT ;

SQL

SQL DML

- Interactive SQL has many powerful queries. An important set of such queries is given below
  - INSERT INTO
  - UPDATE-SET-WHERE
  - DELETE-FROM
  - SELECT-FROM
  - SELECT-FROM-WHERE
  - SELECT-FROM-GROUP BY
  - SELECT-FROM-ORDER BY

SQL

- The INSERT Command
  - To insert a single tuple:

    INSERT INTO <relation>

    VALUES (<list of values>);
  - We may add to the relation name a list of attributes.
  - There are two reasons to do so:
    1. We forget the standard order of attributes for the relation.
    2. We don’t have values for all attributes, and we want the system to fill in missing components with NULL or a default value.

SQL

- Example

```
INSERT INTO EMPLOYEE
VALUES (‘Richard’,’K’,’Marini’,’653298653’,1962-12-30’, ’98 Oak
  Forest,TX’,’M’,37000,’987654321’,4);

INSERT INTO EMPLOYEE (FNAME,LNAME,DNO,SSN)
VALUES (‘Richard’,’Marini’,4,’653298653’);
```

SQL

- Inserting Many Tuples
  - We may insert the entire result of a query into a relation, using the form:
    INSERT INTO <relation>
    (<subquery>);

SQL

- Example

```
INSERT INTO book_of_author
  SELECT id,title,authors_name,publisher
  FROM book
  WHERE author='XYZ';
```

SQL

- The DELETE Command
  - To delete tuples satisfying a condition from some relation:

    DELETE FROM <relation>

    WHERE <condition>;

SQL

- Example

```
DELETE FROM EMPLOYEE
WHERE LNAME='Brown';

DELETE FROM EMPLOYEE;
```

SQL

- The UPDATE Command

  To change certain attributes in certain tuples of a relation:

  UPDATE <relation>

  SET <list of attribute assignments>

  WHERE <condition on tuples>;

SQL

- Example

```
UPDATE  PROJECT
SET     PLOCATION='Bellaire', DNUM=5
WHERE   PNUMBER=10;
```